

ActividadPSP + DIGITALIZACIÓN + SOSTENIBILIDAD, parte de la API

Pablo Martin Tomasi

CampusFP

FECHA ENTREGA: Domingo 25 de enero de 2026

ÍNDICE

Indicador.java.....	2
IndicadorRepositorio.java.....	4
IServicioIndicador.java	5
ServicioIndicador.java	6
IndicadorControlador.java	8
REST Client	11
Listar todos los indicadores	11
Agregar un indicador	12
Eliminar un indicador	13
Actualizar un indicador	13
Filtrar por categoría.....	14
Filtrar por tipo	15
Filtrar por categoría y tipo.....	16

Indicador.java

```
package com.example.digsos.jpa;

import com.fasterxml.jackson.annotation.JsonFormat;
import jakarta.persistence.*;

import java.time.LocalDate;

@Entity//entidad
@Table(name = "Indicador")
public class Indicador {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)//sirve para
    que pueda funcionar de mejor forma el id con auto incremental

    //atributos
    private Integer id;
    private String tipo;
    private String categoria;
    private String nombre;
    private String descripcion;
    private String valor;
    private String unidad;
    @JsonFormat(pattern = "yyyy-MM-dd")//lo tengo puesto porque me
    salia un error al agregar un indicador
    private LocalDate fecha;
    private String ambito;

    //constructor vacio
    public Indicador() {}

    //constructor
    public Indicador(Integer id, String tipo, String categoria,
    String nombre, String descripcion, String valor, String unidad,
    LocalDate fecha, String ambito) {
        this.id = id;
        this.tipo = tipo;
        this.categoria = categoria;
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.valor = valor;
        this.unidad = unidad;
        this.fecha = fecha;
        this.ambito = ambito;
    }

    //Ponemos los get y set
    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }
}
```

```

    }

    public String getCategoria() {
        return categoria;
    }
    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }
    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public String getValor() {
        return valor;
    }
    public void setValor(String valor) {
        this.valor = valor;
    }

    public String getUnidad() {
        return unidad;
    }
    public void setUnidad(String unidad) {
        this.unidad = unidad;
    }

    public LocalDate getFecha() {
        return fecha;
    }
    public void setFecha(LocalDate fecha) {
        this.fecha = fecha;
    }

    public String getAmbito() {
        return ambito;
    }
    public void setAmbito(String ambito) {
        this.ambito = ambito;
    }
}

```

En esta clase definimos la clase de indicador, y declaramos que esta clase es una entidad. Con todos los atributos que vienen de una base de datos de la tabla de indicador. Además, decimos que la primary key es un ID, y se precisa de que es auto incremental. También cuando estaba haciendo pruebas de añadir un indicador, mediante REST Client, me daba un error de que no pillaba el json, y antes del atributo de fecha, digo que se ponga en formato JSON. Para que lo pueda sacar bien

IndicadorRepositorio.java

```
package com.example.digsos.repositorios;

import com.example.digsos.jpa.Indicador;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface IndicadorRepositorio extends JpaRepository<Indicador, Integer> {
    //esto sirve para los filtros
    List<Indicador> findByCategoria(String categoria); //filtrar por categoria
    List<Indicador> findByTipo(String tipo); //filtrar por tipo
    List<Indicador> findByCategoriaAndTipo(String categoria, String tipo); //filtrar por categoria y tipo

    @Query("SELECT i.tipo, COUNT(i) FROM Indicador i GROUP BY i.tipo")
    List<Object[]> resumenPorTipo(); //sentencia sql, con Group By. Se vera en pantalla cuantos indicadores ahy por Digitalizacion y Sostenibilidad
}
```

Esta es una interfaz, donde si en un caso, aquí se puede poner algunas sentencias sql. Y también está también “@Query” para poder ver cuantos indicadores hay por tipo

IServicioIndicador.java

```
package com.example.digsos.servicios;

import com.example.digsos.jpa.Indicador;
import java.util.List;

public interface IServicioIndicador {
    //metodos para poder efectuar algunas peticiones
    List<Indicador> listarIndicador();//seleccionar todos los
    indicadores

    Indicador obtenerIndicador(int id);//obtener un indicador por id

    Indicador guardarIndicador(Indicador indicador);//guardar el
    indicador cuando se agrega o se modifica los datos

    void eliminarIndicador(int id);//eliminar el indicador por el ID

    List<Indicador> filtrarPorCategoria(String categoria);//filtrar
    los indicadores por categoria

    List<Indicador> filtrarPorTipo(String tipo);//filtrar los
    indicadores por tipo

    List<Indicador> filtrarPorCategoriaYTipo(String categoria, String
    tipo);//filtrar la categoria y tipo

    List<Object[]> resumenPorTipo();//método para la sentencia sql de
    resumen por tipo
}
```

En esta interfaz, definimos se define todos los métodos que nos van a servir en la clase de ServicioIndicador.java. Y estos métodos nos sirven en general para hacer el CRUD y también tener los métodos de los filtros. Además de eso, esta también el método para poder ver los indicadores por tipo

ServicioIndicador.java

```
package com.example.digsos.servicios;

import com.example.digsos.jpa.Indicador;
import com.example.digsos.repositorios.IndicadorRepositorio;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

//se indica de que esta clase es un servicio
@Service
public class ServicioIndicador implements IServicioIndicador {
    //repositorio para poder acceder a la base de datos
    private final IndicadorRepositorio indicadorRepositorio;

    //se ponene las dependencias mediante el constructor
    @Autowired
    public ServicioIndicador(IndicadorRepositorio
indicadorRepositorio) {
        this.indicadorRepositorio = indicadorRepositorio;
    }

    //devuelve la lista entera de los indicadores
    @Override
    public List<Indicador> listarIndicador() {
        return indicadorRepositorio.findAll();
    }

    //se obtiene el indicador por su id
    @Override
    public Indicador obtenerIndicador(int id) {
        return indicadorRepositorio.findById(id).orElse(null);
    }

    //se guarda los datos del indicadore, se utiliza para cuando se
    agrega un nuevo inidicador o se actualiza un indicadore
    @Override
    public Indicador guardarIndicador(Indicador indicador) {
        return indicadorRepositorio.save(indicador);
    }

    //elimina el indicador del ID ingresado
    @Override
    public void eliminarIndicador(int id) {
        indicadorRepositorio.deleteById(id);
    }

    //se filtra los indicadores, mediante las categorias
    @Override
    public List<Indicador> filtrarPorCategoria(String categoria) {
        return indicadorRepositorio.findByCategoria(categoria);
    }

    //se filtra los indicadores, mediante los tipos
    @Override
    public List<Indicador> filtrarPorTipo(String tipo) {
        return indicadorRepositorio.findByTipo(tipo);
    }
}
```

```

        //se filtra los indicadores, mediante las categorias y filtros
        @Override
        public List<Indicador> filtrarPorCategoriaYTipo(String categoria,
String tipo) {
            return indicadorRepositorio.findByCategoriaAndTipo(categoria,
tipo);
        }

        //se obtiene el resumen por tipo
        @Override
        public List<Object[]> resumenPorTipo() {
            return indicadorRepositorio.resumenPorTipo();
        }
    }
}

```

En esta clase se indica que es un servicio con `@Service`. Y usando el `@Autowired` se inyectan las dependencias necesarias y se sobrescriben los métodos necesarios con `@Override` para poder efectuar de manera correcta el CRUD y los filtros

IndicadorControlador.java

```
package com.example.digsos.controladores;

import com.example.digsos.jpa.Indicador;
import com.example.digsos.servicios.IServicioIndicador;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
//end point
@RequestMapping("/indicadores")
//sirve para que se pueda acceder a la api, desde cualquier punto
@CrossOrigin(origins = "*")
public class IndicadorControlador {
    private final IServicioIndicador servicioIndicador;

    @Autowired
    public IndicadorControlador(IServicioIndicador servicioIndicador)
    {
        this.servicioIndicador = servicioIndicador;
    }

    //se usa para poder listar todos los indicadores
    @GetMapping("/todos")
    public List<Indicador> todos() {
        return servicioIndicador.listarIndicador();
    }

    //sirve para poder añadir un nuevo indicador
    @PostMapping("/nuevo")
    public Indicador agregarIndicador(@RequestBody Indicador i) {
        servicioIndicador.guardarIndicador(i);
        return i;
    }

    //sirve para poder eliminar un indicador
    @DeleteMapping("/eliminar/{id}")
    public void eliminarIndicador(@PathVariable int id) {
        servicioIndicador.eliminarIndicador(id);
    }

    //sirve para poder actualizar un indicador
    @PutMapping ("/editar/{id}")
    public Indicador editarIndicador(@PathVariable Integer id,
    @RequestBody Indicador i) {
        i.setId(id);
        return servicioIndicador.guardarIndicador(i);
    }

    //filtro para poder ver solo los indicadores de la categoria
    elegida
    @GetMapping("/categoria")
    public List<Indicador> filtroCategoria(@RequestParam String
    categoria) {
```

```

        return servicioIndicador.filtrarPorCategoria(categoria);
    }

    //sirve para poder ver solo los indicadores del tipo elegido
    @GetMapping("/tipo")
    public List<Indicador> filtroTipo(@RequestParam String tipo) {
        return servicioIndicador.filtrarPorTipo(tipo);
    }

    //filtro para poder ver la categoria y filtro elegido
    @GetMapping("/categoria-tipo")
    public ResponseEntity<?> filtroCategoriaYTipo(@RequestParam String
categoria, @RequestParam String tipo) {
        List<Indicador> resultados =
servicioIndicador.filtrarPorCategoriaYTipo(categoria, tipo);

        //si en un caso, una categoria no tiene un tipo buscado o vice
versa, se indica de que no se ha encontrado
        if (resultados.isEmpty()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of(
                    "error", "No se encontraron indicadores
con los filtros indicados",
                    "categoria", categoria,
                    "tipo", tipo
                ));
        }

        return ResponseEntity.ok(resultados);
    }

    //nos sirve para poder ver el resumen por tipo
    @GetMapping("/resumenTipo")
    public List<Object[]> resumenPorTipo() {
        return servicioIndicador.resumenPorTipo();
    }
}

```

Esta clase es la que controla el CRUD en general. Ya que se indica al principio que esta clase es un RestController, que también para acceder a los datos de la base de datos es con RequestMapping con /indicadores, y el CrossOrigin nos sirve para poder acceder a la api sin ninguna restricción.

Luego se indica que se usa la clase de IServicioIndicador. Luego se usa los métodos http, para poder hacer el CRUD. Los métodos son estos:

- GetMapping
 - Este método se utiliza 5 veces
 - Se utiliza ya para poder listar todos los indicadores a la vez
 - Se utiliza para poder listar los indicadores con la misma categoría (filtro)
 - Se utiliza para poder listar los indicadores del mismo tipo (filtro)
 - Se utiliza para poder listar los indicadores de la misma categoría y mismo tipo (filtro)

- Se utiliza para poder obtener el resumen de los indicadores por tipo (filtro)
- PostMapping
 - Este método se utiliza cuando se va a añadir un indicador nuevo a la api
- DeleteMapping
 - Este método se utiliza cuando se va a eliminar un indicador de la api
- PutMapping
 - Este método se utiliza para actualizar los datos de un indicador de la api

REST Client

Para poder verificar que los métodos del CRUD funcionen bien. Y se puede usar esta extensión la de REST Client, para poder ver que funcione todo correctamente

Listar todos los indicadores

The screenshot displays the REST Client extension interface. At the top, the URL bar shows 'localhost:8081/indicadores/todos' with a 'GET' method selected. Below this, there's a section for headers with a dropdown menu showing 'Headers (1)' and an 'Add Header' button. A text area for the request body is labeled 'Enter JSON or text body'. A 'Send Request' button is located at the bottom of the request section.

The response section shows a 'Response' status with a '200' status code, 'GET' method, and '646ms' response time. Below this, there are tabs for 'JSON', 'Body', and 'Headers'. The 'JSON' tab is selected, displaying a JSON array of three objects. The JSON is formatted with line numbers 1 through 26.

```
1 [
2   {
3     "id": 1,
4     "tipo": "Digitalización",
5     "categoria": "Procesos",
6     "nombre": "Procesos digitalizados",
7     "descripcion": "Número de procesos internos que han sido digitalizados.",
8     "valor": "8",
9     "unidad": "procesos",
10    "fecha": "2024-01-15",
11    "ambito": "Empresa"
12  },
13  {
14    "id": 2,
15    "tipo": "Digitalización",
16    "categoria": "Tecnología",
17    "nombre": "Uso de firma digital",
18    "descripcion": "Implantación de firma digital en trámites internos.",
19    "valor": "Sí",
20    "unidad": "boolean",
21    "fecha": "2024-01-20",
22    "ambito": "Empresa"
23  },
24  {
25    "id": 3,
26    "tipo": "Digitalización",
```

Agregar un indicador

POST

localhost:8081/indicadores/nuevo

Headers (1)

+ Add Header

Name	Value	Actions
Content-Type	application/json	

```
{
  "tipo": "Sostenibilidad",
  "categoria": "Procesos",
  "nombre": "Procesos digitalizados",
  "descripcion": "Número de procesos internos que han sido digitalizados.",
  "valor": "19",
  "unidad": "procesos",
  "fecha": "2024-01-15",
  "ambito": "Empresa"
}
```

Send Request

Response 200 POST 274ms

JSON Body Headers

```
1 {
2   "id": 82,
3   "tipo": "Sostenibilidad",
4   "categoria": "Procesos",
5   "nombre": "Procesos digitalizados",
6   "descripcion": "Número de procesos internos que han sido digitalizados.",
7   "valor": "19",
8   "unidad": "procesos",
9   "fecha": "2024-01-15",
10  "ambito": "Empresa"
11 }
```

Eliminar un indicador

DELETE

localhost:8081/indicadores/eliminar/6

Headers (1)

+ Add Header

Enter JSON or text body

Send Request

Response 200 DELETE 66ms

Body Headers

No response yet

Actualizar un indicador

Ctrl+B PUT localhost:8081/indicadores/editar/5

PUT

localhost:8081/indicadores/editar/5

Headers (1)

+ Add Header

Name	Value	Actions
Content-Type	application/json	

```
{
  "tipo": "Sostenibilidad",
  "categoria": "Prueba",
  "nombre": "Procesos digitalizados",
  "descripcion": "Número de procesos internos que han sido digitalizados.",
  "valor": "19",
  "unidad": "procesos",
  "fecha": "2024-01-15",
  "ambito": "Prueba",
  "id": "5"
}
```

Send Request

Response 200 PUT 73ms

JSON Body Headers

```
1 {
2   "id": 5,
3   "tipo": "Sostenibilidad",
4   "categoria": "Prueba",
5   "nombre": "Procesos digitalizados",
6   "descripcion": "Número de procesos internos que han sido digitalizados.",
7   "valor": "19",
8   "unidad": "procesos",
9   "fecha": "2024-01-15",
10  "ambito": "Prueba"
11 }
```

Filtrar por categoría

Ctrl+B GET localhost:8081/indicadores/categoria?categoria=Procesos

GET

localhost:8081/indicadores/categoria?categoria=Procesos

Headers (1)

+ Add Header

Enter JSON or text body

Send Request

Response 200 GET 456ms

JSON Body Headers

```
1 [
2   {
3     "id": 1,
4     "tipo": "Digitalización",
5     "categoria": "Procesos",
6     "nombre": "Procesos digitalizados",
7     "descripcion": "Número de procesos internos que han sido digitalizados.",
8     "valor": "8",
9     "unidad": "procesos",
10    "fecha": "2024-01-15",
11    "ambito": "Empresa"
12  },
13  {
14    "id": 4,
15    "tipo": "Digitalización",
16    "categoria": "Procesos",
17    "nombre": "Facturación electrónica",
18    "descripcion": "Uso de facturación electrónica en lugar de facturas en papel.",
19    "valor": "100",
20    "unidad": "%",
21    "fecha": "2024-02-05",
22    "ambito": "Empresa"
23  },
24  {
25    "id": 7,
26    "tipo": "Digitalización",
27    "categoria": "Procesos",
```

Filtrar por tipo

🔑

Ctrl+B

GET

localhost:8081/indicadores/tipo?tipo=Digitalización

GET

localhost:8081/indicadores/tipo?tipo=Digitalización

🔍 Headers (1)

+ Add Header

Enter JSON or text body

🚀 Send Request

Response

200

GET

49ms

JSON

Body

Headers

1 [

2 {

3 "id": 1,

4 "tipo": "Digitalización",

5 "categoria": "Procesos",

6 "nombre": "Procesos digitalizados",

7 "descripcion": "Número de procesos internos que han sido digitalizados.",

8 "valor": "8",

9 "unidad": "procesos",

10 "fecha": "2024-01-15",

11 "ambito": "Empresa"

12 },

13 {

14 "id": 2,

15 "tipo": "Digitalización",

16 "categoria": "Tecnología",

17 "nombre": "Uso de firma digital",

18 "descripcion": "Implantación de firma digital en trámites internos.",

19 "valor": "Si",

20 "unidad": "boolean",

21 "fecha": "2024-01-20",

22 "ambito": "Empresa"

23 },

24 {

25 "id": 3,

26 "tipo": "Digitalización",

27 "categoria": "Datos",

Filtrar por categoría y tipo

Ctrl+B

GET localhost:8081/indicadores/categoria-tipo?categoria=Social&tipo=Sostenibilidad

GET

localhost:8081/indicadores/categoria-tipo?categoria=Social&tipo=Sostenibilidad

Headers (1)

+ Add Header

Enter JSON or text body

Send Request

Response 200 GET 39ms

JSON

Body

Headers

```
1 [
2   {
3     "id": 26,
4     "tipo": "Sostenibilidad",
5     "categoria": "Social",
6     "nombre": "Formación en sostenibilidad",
7     "descripcion": "Número de acciones formativas sobre sostenibilidad.",
8     "valor": "2",
9     "unidad": "acciones",
10    "fecha": "2024-02-15",
11    "ambito": "Empresa"
12  },
13  {
14    "id": 27,
15    "tipo": "Sostenibilidad",
16    "categoria": "Social",
17    "nombre": "Igualdad de género",
18    "descripcion": "Porcentaje de mujeres en puestos de responsabilidad.",
19    "valor": "45",
20    "unidad": "%",
21    "fecha": "2024-02-20",
22    "ambito": "Empresa"
23  },
24  {
25    "id": 32,
26    "tipo": "Sostenibilidad",
27    "categoria": "Social",
```

Resumen por tipo

GET

http://localhost:8081/indicadores/resumenTipo

Headers (1)

+ Add Header

Enter JSON or text body

Send Request

Response

200

GET

344ms

JSON

Body

Headers

1 [

2 [

3 "Digitalización",

4 38

5],

6 [

7 "Sostenibilidad",

8 43

9]

10]